

# FMFUT

Audit technique, manuel et guide de reprise

|         |                        |
|---------|------------------------|
| VERSION | 0.13.3                 |
| ÉTAT    | Prototype local validé |
| AUDIT   | 23 juillet 2026        |

Référence produit et technique destinée à une reprise du projet par une personne accompagnée d'un agent.

*État audité le 23 juillet 2026. Ce document est la référence principale du projet. Il décrit le produit, son architecture, son exploitation, ses tests, ses limites et les travaux nécessaires avant une mise en production.*

## 1. Résumé exécutif

FMFUT est un prototype local de jeu de management football. Il combine :

- une base de 18 405 joueurs transformée depuis un export de type EA FC/FIFA ;
- un constructeur d'équipes et de stratégies ;
- un moteur de match 2D déterministe exécuté côté serveur ;
- un replay interactif côté navigateur ;
- deux laboratoires Monte Carlo avec statistiques collectives, individuelles

et spatiales.

La V0.13.3 est cohérente, compilable et bien testée pour un prototype. Elle n'est toutefois **pas prête pour une exposition publique** sans protections supplémentaires. Les trois freins majeurs sont :

- la licence du dataset est non vérifiée ;
- les routes de simulation coûteuses n'ont ni authentification, ni quota, ni limitation de débit ;
- l'audit npm signale encore une vulnérabilité haute transitive dans sharp.

La dette structurelle principale est concentrée dans `engine.ts` (environ 2 700 lignes), `globals.css` (environ 2 300 lignes) et `PitchCanvas.tsx` (environ 800 lignes). Leur découpage est prioritaire avant d'accélérer fortement le développement produit.

## 2. Périmètre fonctionnel de la V0.13.3

### Équipes - /squad

- création de plusieurs équipes locales ;
- plusieurs stratégies par équipe ;
- emblèmes emoji distincts pour équipes et stratégies ;
- formations 4-3-3, 4-2-3-1 et 4-1-4-1 ;
- recherche paginée parmi 18 405 joueurs ;
- filtres par nom, poste, note minimale et nationalité Coupe du monde ;
- placement par clic et glisser-déposer ;
- échanges terrain-terrain, banc-terrain et terrain-banc ;
- banc de sept joueurs avec emplacements libres ;
- rôles individuels ;
- hauteur, largeur du bloc et style de construction ;
- diagnostic de compatibilité, synergie et équilibre ;
- génération d'une équipe aléatoire respectant les postes ;

- import/export JSON versionné ;
- sauvegarde locale automatique.

## Matches - /squad/match

- choix d'une équipe et d'une stratégie sauvegardées ;
- choix d'une sélection nationale disponible ;
- aperçu des deux formations avant lancement ;
- simulation, replay, commentaires et chronologie complète ;
- conservation locale du dernier résultat pendant la session.

## Simulateur d'équipe - /squad/tests

- séries de 10 à 100 matchs ;
- seed déterministe ;
- comparaison optionnelle de deux stratégies ;
- distributions de résultats ;
- statistiques collectives et individuelles des deux équipes ;
- heatmaps par équipe, joueur, poste, période et phase de possession ;
- métriques de hauteur, largeur, profondeur et volatilité du bloc.

## Partie rapide - /

- choix libre de deux sélections, y compris la même sélection des deux côtés ;
- couleurs de maillot automatiques par nation ;
- aperçu actualisé des deux onze avant simulation ;
- seed modifiable ;
- replay complet, score, statistiques et notifications d'après-match.

## Monte Carlo Lab - /analytics

- 10 à 300 matchs par série ;
- inversion alternée des côtés afin de neutraliser l'avantage du premier côté ;
- statistiques collectives et individuelles ;
- heatmaps et métriques spatiales ;
- sensibilité automatique aux six statistiques ;
- expérience sur les rôles ;
- micro-benchmarks isolés ;
- conservation des réglages et du dernier rapport pendant la session.

# 3. Démarrage rapide

## Prérequis

- Node.js 22.13 ou supérieur ;

- npm ;
- Python 3.10 ou supérieur pour reconstruire et tester le dataset ;
- SQLite avec FTS5, fourni via better-sqlite3 par défaut.

## Installation et développement

```
cd simulator
npm install
npm run dev
```

Ouvrir `http://localhost:3000`.

## Validation complète

```
cd simulator
npm run test:all
npm run build
```

## Scripts utiles

| Commande                                                  | Usage                               |
|-----------------------------------------------------------|-------------------------------------|
| <code>npm run dev</code>                                  | serveur de développement            |
| <code>npm run build</code>                                | compilation de production           |
| <code>npm run start</code>                                | exécution du build Next.js          |
| <code>npm run typecheck</code>                            | contrôle TypeScript strict          |
| <code>npm test</code>                                     | suite Vitest                        |
| <code>npm run test:dataset</code>                         | tests Python du pipeline de données |
| <code>npm run test:all</code>                             | TypeScript, Vitest et dataset       |
| <code>npm run test:calibration</code>                     | tests statistiques ciblés           |
| <code>npm run calibrate:match -- 500 --compact</code>     | campagne de calibration             |
| <code>npm run benchmark:data</code>                       | benchmark du repository joueurs     |
| <code>npm run package:data -- --output ./dist/data</code> | packaging contrôlé du dataset       |

## 4. Arborescence et responsabilités

```

fmfut/
  README.md référence de reprise et audit
  fut_manager_game_design_v1.pdf cahier des charges historique
  fut_manager_simulator_v0.9_handoff_report.pdf
  dataset/
    build_players_db.py import et normalisation
    stat_formulas.py calcul des six statistiques
    positions.py mapping des postes
    schema.sql schéma SQLite et FTS5
    players.db base runtime locale
    fut_players.csv source locale
    SOURCE_AND_LICENSE.md statut juridique
    tests/ tests Python
  simulator/
    app/ pages et Route Handlers Next.js
    components/ interface React
    lib/game/ moteur déterministe
    lib/data/ accès SQLite/CSV et recherche
    lib/squad/ équipes, diagnostics et Monte Carlo
    lib/analytics/ calibration et agrégations
    lib/client/ état local des écrans
    tests/ tests Vitest
    scripts/ calibration, benchmark, packaging
    CALIBRATION.md méthodologie statistique détaillée

```

## 5. Architecture d'exécution

```

CSV source
↓ build_players_db.py
SQLite + FTS5
↓ repository serveur
Route Handler Next.js
↓ validation du contrat
simulateMatch(...)
↓ résultat + stats + événements + frames
JSON
↓
React + Canvas 2D

```

Le navigateur ne décide jamais du résultat. Il reçoit un match déjà calculé et interpole les frames du replay. À configuration et seed identiques, le résultat est identique.

La façade publique du moteur est `simulator/lib/game/index.ts`. Les nouveaux consommateurs doivent importer `simulateMatch`, le contrat et les types depuis cette façade. Les imports directs de `engine.ts` sont interdits hors du moteur.

## 6. Moteur de match

### Horloge

- 90 minutes affichées ;
- 360 secondes logiques par défaut ;
- pas physique de 0,2 seconde ;
- sous-pas du ballon de 0,04 seconde ;
- décision IA chaque seconde logique ;
- frame de replay toutes les 0,16 seconde ;
- échantillon spatial chaque seconde.

À vitesse x1, un match dure environ six minutes réelles dans le viewer.

## Modèle actuel

Le moteur gère notamment :

- mouvement avec et sans ballon ;
- forme collective et suivi du ballon ;
- largeur, hauteur et profondeur de bloc ;
- pressing, fatigue et énergie ;
- passes, dribbles, tirs et interceptions physiques ;
- hors-jeu ;
- duels, fautes, cartons et blessures ;
- touches, corners, six mètres, coups francs et penalties ;
- arrêts et relances du gardien ;
- remplacements automatiques ;
- temps additionnel ;
- statistiques collectives et individuelles ;
- replay et analyses spatiales.

## Six statistiques publiques

- vitesse ;
- tir ;
- passe ;
- physique ;
- technique ;
- intelligence.

Les rôles modulent les comportements sans remplacer les statistiques : DEFENSIVE, NORMAL, OFFENSIVE, CREATOR, PRESSING.

## Neutralité des côtés

Le moteur ne donne pas de bonus de domicile. Les campagnes Monte Carlo alternent les côtés à chaque match et réattribuent ensuite les résultats à l'équipe logique d'origine.

# 7. Données joueurs

## État du dataset

- 18 405 joueurs ;
- 160 nationalités ;
- schéma de base version 2 ;
- base SQLite d'environ 26 Mo ;
- CSV source d'environ 10 Mo ;

- statut de licence : `unverified`.

La présence de ces données dans le dépôt ne constitue pas un droit de redistribution ou d'exploitation commerciale. Le packaging public est bloqué par défaut tant que les métadonnées ne déclarent pas une licence vérifiée.

## Ordre de sélection de la source runtime

- `PLAYERS_CSV_PATH`, si défini ;
- `players.db`, si présente ;
- petit CSV de démonstration.

Variables reconnues :

| Variable                      | Effet                                                                               |
|-------------------------------|-------------------------------------------------------------------------------------|
| <code>PLAYERS_DB_PATH</code>  | chemin absolu vers la base SQLite                                                   |
| <code>PLAYERS_CSV_PATH</code> | force une source CSV normalisée                                                     |
| <code>SQLITE_DRIVER</code>    | <code>better-sqlite3</code> par défaut, <code>node:sqlite</code> en repli explicite |

## Sélections nationales

Le catalogue source définit 48 nations. La V0.13.3 ne publie que les sélections pour lesquelles le dataset permet de construire onze titulaires compatibles et sept remplaçants réels. L'audit courant en conserve 35. Aucune fiche fictive Réserve N n'est renvoyée par l'API.

## 8. Contrats HTTP

| Méthode et route                             | Rôle                          | Validation / limites                    |
|----------------------------------------------|-------------------------------|-----------------------------------------|
| GET <code>/api/players</code>                | recherche paginée             | Zod, 100 résultats maximum              |
| GET <code>/api/players/metadata</code>       | métadonnées de source         | lecture seule                           |
| GET <code>/api/players/benchmarks</code>     | quantiles par poste           | cache processus                         |
| GET <code>/api/squad/bootstrap</code>        | équipe de démonstration       | lecture seule                           |
| GET <code>/api/squad/opponents</code>        | sélections complètes          | cache processus                         |
| GET <code>/api/squad/random</code>           | équipe aléatoire déterministe | seed libre                              |
| POST <code>/api/squad/preview</code>         | 10 à 100 simulations          | Zod, cache LRU 100 entrées / 5 min      |
| POST <code>/api/matches/simulate</code>      | match et replay               | Zod, 60 à 900 secondes logiques         |
| POST <code>/api/analytics/monte-carlo</code> | laboratoire complet           | 10 à 300 matchs, validation à renforcer |

Exemple minimal de match :

```
{
  "seed": "demo-42"
}
```

Le contrat public est versionné 1.0.0. Une `TeamSelection` contient le nom, la formation, onze IDs titulaires, le banc, les rôles et les tactiques.

## 9. Persistance client

La V0.13 ne possède ni compte ni stockage serveur des effectifs.

- équipes et stratégies : `localStorage`, espace `fmfut:squad-workspace:v2` ;
- thème : `localStorage` ;
- réglages et résultats d'écrans : mémoire du module + `sessionStorage` ;
- résultats trop volumineux pour le quota navigateur : conservés en mémoire pendant la navigation, mais pas après rechargement complet.

Conséquences :

- les données sont propres au navigateur et à l'appareil ;
- vider les données du site supprime les équipes ;
- aucune synchronisation multi-appareil ;
- aucune confidentialité forte : tout script exécuté sur la même origine peut lire ces données.

## 10. Tests et garanties actuelles

La suite couvre :

- déterminisme par seed ;
- contrat HTTP et validation des compositions ;
- cohérence score/événements ;
- absence de NaN et d'infinis ;
- joueurs et ballon dans les limites du terrain ;
- règles de sortie, reprises et buts physiques ;
- hors-jeu, remplacements et statistiques individuelles ;
- neutralité et matchs miroir ;
- hauteur et largeur du bloc ;
- heatmaps et agrégations spatiales ;
- recherche SQLite, cache et métadonnées ;
- import du dataset ;
- génération et échanges du Squad Builder ;
- catalogue national sans joueurs fictifs ;
- aperçu des formations et symétrie des deux onze ;
- calibration statistique et micro-benchmarks.

La suite ne couvre pas encore :

- parcours de navigateur de bout en bout ;
- régressions visuelles multi-résolutions ;
- accessibilité automatisée ;
- tests de charge et concurrence ;
- fuzzing des contrats et imports JSON ;
- reprise après crash ou dépassement de quota navigateur ;
- compatibilité multi-navigateurs réelle.

# 11. Audit de sécurité

## Synthèse des risques

| Priorité        | Risque                                                   | État                   | Action recommandée                                                         |
|-----------------|----------------------------------------------------------|------------------------|----------------------------------------------------------------------------|
| Bloquant public | licence du dataset non vérifiée                          | confirmé               | remplacer/licencier la source avant diffusion                              |
| Haute           | déni de service CPU via simulations publiques            | confirmé               | authentification, quotas, rate limit, file de jobs                         |
| Haute           | vulnérabilité transitive sharp < 0.35.0                  | signalée par npm audit | suivre la correction compatible, supprimer l'optimisation image si inutile |
| Moyenne         | validation incomplète du Monte Carlo Lab                 | confirmé               | schéma Zod strict commun à toutes les routes                               |
| Moyenne         | messages d'erreur internes renvoyés au client            | confirmé               | journaliser côté serveur, réponse publique générique                       |
| Moyenne         | aucune limite explicite de taille du JSON                | confirmé               | limite proxy et lecture bornée du corps                                    |
| Moyenne         | absence de CSP et d'en-têtes de défense explicites       | confirmé               | CSP, nosniff, politique de référent et permissions                         |
| Moyenne         | stockage navigateur lisible par tout script de l'origine | inhérent               | minimiser, versionner, sauvegarder/exporter, éviter les secrets            |
| Faible          | endpoint aléatoire accepte une seed non bornée           | confirmé               | limiter longueur et caractères                                             |
| Faible          | cache mémoire par processus seulement                    | confirmé               | cache distribué ou jobs persistants en production                          |

## Dépendances

Next.js a été mis à jour de 16.2.10 à 16.2.11 pendant l'audit afin de corriger plusieurs avis de sécurité du framework. Après cette mise à jour, `npm audit --omit=dev` signale encore deux entrées hautes liées à une seule cause transitive : sharp < 0.35.0 et son libvips embarqué (GHSA-f88m-g3jw-g9cj).

La V0.13.3 n'utilise pas next/image ni de traitement d'images utilisateur, ce qui réduit la surface d'exploitation, sans supprimer le problème de chaîne d'approvisionnement. Ne pas appliquer de rétrogradation majeure automatique de Next.js uniquement pour satisfaire npm audit ; tester une version de sharp corrigée ou attendre une résolution compatible officielle.

## Entrées et injection

- les requêtes SQLite utilisent des paramètres, ce qui protège les valeurs de recherche contre l'injection SQL ;
- la requête FTS échappe les guillemets et assemble des tokens contrôlés ;
- React échappe les noms affichés et aucun dangerouslySetInnerHTML n'est présent ;
- les routes match, joueurs et aperçu utilisent Zod ;
- la route analytique fait encore un cast TypeScript du JSON et doit adopter le même schéma runtime strict ;
- les chemins PLAYERS\_DB\_PATH et PLAYERS\_CSV\_PATH sont des réglages

administrateur : ils ne doivent jamais être contrôlés par un utilisateur.

## Disponibilité

Le Monte Carlo Lab peut déclencher une baseline, six expériences de sensibilité, une expérience de rôles et 10 000 micro-échantillons dans une seule requête. À 300 matchs, cet endpoint bloque longtemps un worker Node. Sans contrôle d'accès, quelques requêtes concurrentes peuvent saturer CPU et mémoire.

Avant exposition :

- authentifier l'appelant ;
- limiter les séries synchrones ;
- introduire une file de jobs avec annulation et progression ;
- appliquer quotas par compte et par IP ;
- imposer délais et limites mémoire ;
- mettre en cache les résultats déterministes par empreinte de configuration.

## Authentification et autorisation

Il n'existe aucun compte, rôle ou permission. C'est conforme au prototype local, mais incompatible avec des effectifs persistés ou un service public. Les routes ne doivent pas être considérées privées parce qu'elles ne sont pas liées dans l'interface.

# 12. Audit de qualité et maintenabilité

## Points solides

- TypeScript strict ;
- moteur déterministe et façade publique explicite ;
- logique serveur séparée du rendu Canvas ;
- contrats Zod sur les routes principales ;
- SQLite paramétré et FTS5 ;
- cache LRU borné ;
- dataset restructurable avec empreinte SHA-256 ;
- tests de caractérisation protégeant le comportement du moteur ;
- statistiques individuelles et spatiales riches ;
- séparation progressive du moteur en modules de règles ;
- aucune dépendance UI lourde.

## Dette prioritaire

- `engine.ts` **reste monolithique**. Il mélange boucle, décisions, actions, déplacements, possession, horloge et émission des événements.
- `globals.css` **accumule les versions**. Plusieurs règles portant les mêmes sélecteurs dépendent de l'ordre de cascade.
- `**PitchCanvas.tsx` mélange rendu, lecture, contrôles, commentaires et interpolation.\*\* Il devrait être découpé en viewer, contrôleur et panneaux.

- **Les agrégations Monte Carlo sont synchrones.** Elles occupent le thread du serveur et construisent de grandes structures en mémoire.
- **Les types API sont partagés manuellement.** Un schéma source unique devrait produire validation et types.
- **\*\*Le catalogue national construit encore des réservistes avant de filtrer l'équipe incomplète.\*\*** Le résultat public est correct mais l'algorithme peut être simplifié en validation sans objets synthétiques.
- **Les README historiques étaient désynchronisés.** Ce document devient la source canonique ; les changelogs restent des archives.

## 13. Limites produit et simulation

- pas de comptes ni de profils ;
- pas de sauvegarde serveur ;
- pas de marché, prix, contrats ou économie ;
- pas de clubs persistants ni compétitions ;
- seulement trois formations ;
- sélections nationales précomposées en 4-3-3 ;
- remplacements automatiques uniquement ;
- aucune modification tactique pendant le replay ;
- pas de moteur de saison, calendrier, classement ou progression ;
- blessures et suspensions retournées par un match mais non persistées ;
- pas de météo, arbitre, moral, pied fort exploité, taille ou âge dans le gameplay courant ;
- animation abstraite, sans collisions corporelles complètes ;
- calibrage basé sur le comportement interne et non validé contre un corpus professionnel de matchs réels ;
- noms, notes et nationalités héritent des biais et erreurs de la source ;
- 35 sélections seulement sont complètes avec le dataset actuel.

### Limite connue à corriger

La fin d'une mi-temps peut intervenir exactement lorsque le chrono atteint la limite. Une évolution devrait attendre la fin de l'action en cours selon la position et l'état du ballon, avec une borne maximale pour éviter une période infinie.

## 14. Performance et capacité

- tous les joueurs sont chargés et conservés en mémoire serveur pour le moteur ;
- la recherche paginée reste effectuée directement par SQLite ;
- les replays complets sont volumineux ;
- sessionStorage peut refuser un replay trop grand, auquel cas la mémoire du

module assure seulement la navigation courante ;

- le catalogue d'adversaires est caché par processus ;
- le cache de recherche est limité à 250 entrées pendant 60 secondes ;
- le cache d'aperçu est limité à 100 rapports pendant cinq minutes ;
- aucune mesure de charge multi-utilisateur n'a été réalisée.

Pour la production, mesurer séparément : temps CPU par match, taille JSON du replay, mémoire par série Monte Carlo, latence SQLite, concurrence et coût du Canvas sur appareils modestes.

## 15. Procédure de reprise par une personne et un agent

- Lire ce README, CALIBRATION.md et dataset/SOURCE\_AND\_LICENSE.md.
- Vérifier `git status` pour ne pas écraser des changements locaux.
- Installer dans `simulator/` et lancer `npm run test:all`.
- Lancer `npm run build` avant toute modification structurante.
- Utiliser `lib/game/index.ts` comme frontière du moteur.
- Ajouter un test de caractérisation avant de modifier une règle centrale.
- Pour une évolution statistique, comparer plusieurs familles de seeds et

utiliser les expériences appariées.

- Pour une évolution de données, reconstruire la base et vérifier les métadonnées/licences.

- Pour une évolution UI, tester clair/sombre, 16:9, laptop, mobile et zoom clavier.

- Mettre à jour ce README, la version du package et le PDF d'audit à chaque clôture de version majeure ou mineure.

## 16. Checklist avant merge ou livraison

- ☐ composition et contrats restent rétrocompatibles ou sont migrés ;
- ☐ aucun import applicatif direct de `engine.ts` ;
- ☐ tests unitaires ajoutés pour la nouvelle règle ;
- ☐ `npm run test:all` réussit ;
- ☐ `npm run build` réussit ;
- ☐ déterminisme vérifié si le moteur change ;
- ☐ calibration exécutée si probabilités ou statistiques changent ;
- ☐ clair/sombre et responsive contrôlés si l'interface change ;
- ☐ aucune donnée ou licence non vérifiée ajoutée à un package public ;
- ☐ `npm audit --omit=dev` examiné sans correction majeure automatique ;
- ☐ documentation et numéro de version mis à jour.

## 17. Étapes recommandées à court terme

### Avant toute publication

- résoudre la licence des données ;
- protéger les endpoints coûteux ;
- corriger ou isoler la dépendance sharp ;
- uniformiser la validation Zod et les erreurs API ;
- ajouter CSP et en-têtes de sécurité ;
- mettre en place CI, tests E2E et tests de charge.

### Stabilisation technique

- découper engine.ts en orchestration, décisions, possession et horloge ;
- modulariser PitchCanvas ;
- remplacer l'empilement CSS par des feuilles organisées par surface ;
- créer un job runner Monte Carlo avec progression ;
- définir un schéma partagé versionné pour toutes les API ;
- ajouter métriques, logs structurés et traces de performance.

## 18. Grandes branches possibles pour la suite

### Branche A - jeu de management persistant

Comptes, effectifs serveur, saisons, calendrier, blessures, suspensions, progression, objectifs et sauvegardes multi-appareils.

### Branche B - profondeur tactique et moteur

Davantage de formations, consignes par ligne et joueur, changements en match, phases de jeu, coups de pied arrêtés configurables et moteur physique enrichi.

### Branche C - squad builder décisionnel

Optimisation sous contraintes, recommandations expliquées, comparaison de joueurs, recherche de complémentarités, scénarios et assistants de recrutement.

### Branche D - compétition et économie

Marché, prix, contrats, budgets, récompenses, tournois et équilibre économique.

### Branche E - plateforme analytique

Jobs distribués, grandes campagnes Monte Carlo, calibration sur données réelles, exports, tableaux comparatifs et observabilité scientifique.

### Branche F - expérience visuelle

Accessibilité, mobile, narration du match, son, animations, personnalisation et tests visuels automatisés.

## 19. Références internes

- `simulator/CALIBRATION.md` : protocole statistique ;
- `dataset/README.md` : construction de la base ;
- `dataset/SOURCE_AND_LICENSE.md` : droits et provenance ;
- `fut_manager_game_design_v1.pdf` : vision produit initiale ;
- `fut_manager_simulator_v0.9_handoff_report.pdf` : historique V0.9 ;
- `simulator/CHANGELOG_V0.*.md` : archives des versions précédentes.